

Investigative Reversing 0 | picoCTF Writeup

by Alexander Sapo

Category: Forensics

Difficulty: Hard

Challenge

We are given a binary and an image. The goal is to find the hidden flag.

- `mystery` — ELF 64-bit executable
- `mystery.png` — PNG image

The hints suggest using both **forensics and reversing**, with a hex editor potentially being useful.

Investigation

I first checked the file types:

```
mystery.png: PNG image data, 1411 x 648, 8-bit/color RGB, non-interlaced
```

```
mystery: ELF 64-bit LSB pie executable, x86-64, dynamically linked,
not stripped
```

Since the PNG looked normal, I opened it in a hex editor and inspected the **end of the file**.

The screenshot displays a Linux terminal session where a file named 'mystery.png' is being analyzed. The user first uses the 'zsteg' command to extract hidden data from the PNG file, resulting in two lines of hexadecimal output. Next, the 'file' command is used to identify the file's format, revealing it to be an ELF 64-bit LSB pie executable for x86-64 architecture. In the background, another terminal window is visible, showing the process of processing symbol tables.

```
conan0s4@SolarisFortress:/media/sf_Bridge/ctf
```

```
Session Actions Edit View Help
```

```
(conan0s4@SolarisFortress)-[/media/sf_Bridge/ctf]
$ stegsnow mystery.png
```

```
(conan0s4@SolarisFortress)-[/media/sf_Bridge/ctf]
$ zsteg mystery.png
[?] 26 bytes of extra data after image end (IEND), offset = 0x1e873
extra data:
00000000: 70 69 63 6f 43 54 4b 80  6b 35 7a 73 69 64 36 71  |picoCTK.KSzsid6q|
00000010: 5f 36 30 39 39 64 66 62  6d 7d                                |_G099dfdb|
imagedata .. text: "PPP000000000000"
        ^-----^
(conan0s4@SolarisFortress)-[/media/sf_Bridge/ctf]
$ file mystery
mystery: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 3.2.0, BuildID[sha1]=34b772a4f30594e2f30ac431c72667c3e10fa3e9, not stripped
```

```
(conan0s4@SolarisFortress)-[/media/sf_Bridge/ctf]
```

Processing symbol tables...

Cancel

There was extra data after the normal PNG data. The bytes at the end were also visible as ASCII characters, which immediately suggested that something had been **appended**

to the image.

The next question was: **what exactly was appended, and was it modified somehow?**

That's where the binary comes in.

Reversing the Binary

I opened `mystery` in a decompiler. The full relevant `main()` function was:

```
void main(void)

{
    FILE *__stream;
    FILE *__stream_00;
    size_t sVar1;
    long in_FS_OFFSET;
    int local_54;
    int local_50;
    char local_38 [4];
    char local_34;
    char local_33;
    char local_29;
    long local_10;

    local_10 = *(long *) (in_FS_OFFSET + 0x28);
    __stream = fopen("flag.txt","r");
    __stream_00 = fopen("mystery.png","a");

    if (__stream == (FILE *)0x0) {
        puts("No flag found, please make sure this is run on the server");
    }

    if (__stream_00 == (FILE *)0x0) {
        puts("mystery.png is missing, please run this on the server");
    }

    sVar1 = fread(local_38,0x1a,1,__stream);

    if ((int)sVar1 < 1) {
        /* WARNING: Subroutine does not return */
        exit(0);
    }
}
```

```

puts("at insert");

fputc((int)local_38[0],__stream_00);
fputc((int)local_38[1],__stream_00);
fputc((int)local_38[2],__stream_00);
fputc((int)local_38[3],__stream_00);
fputc((int)local_34,__stream_00);
fputc((int)local_33,__stream_00);

for (local_54 = 6; local_54 < 0xf; local_54 = local_54 + 1) {
    fputc((int)(char)(local_38[local_54] + '\x05'),__stream_00);
}

fputc((int)(char)(local_29 + -3),__stream_00);

for (local_50 = 0x10; local_50 < 0x1a; local_50 = local_50 + 1)
{
    fputc((int)local_38[local_50],__stream_00);
}

fclose(__stream_00);
fclose(__stream);

if (local_10 != *(long *)(in_FS_OFFSET + 0x28)) {
    /* WARNING: Subroutine does not return */
    __stack_chk_fail();
}

return;
}

```

The important parts are:

```

__stream = fopen("flag.txt","r");
__stream_00 = fopen("mystery.png","a");

```

The program reads the flag from `flag.txt` and opens the PNG in **append mode**.

Then:

```

sVar1 = fread(local_38,0x1a,1,__stream);

```

0x1a is 26 in decimal, meaning the original flag is **26 bytes long**.

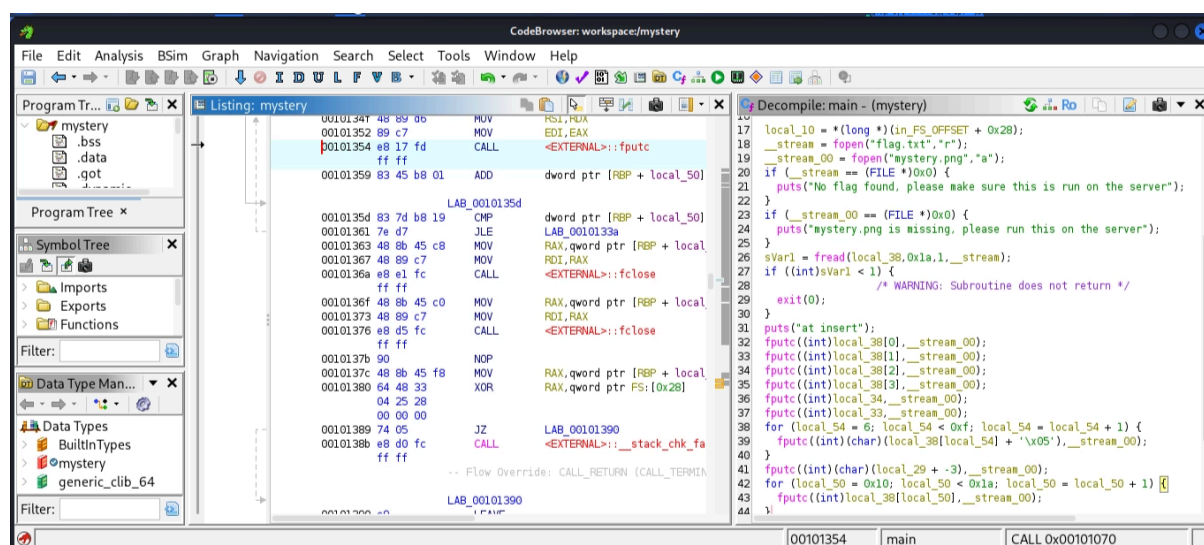
The program then writes those 26 bytes to the end of the PNG, but not all of them are written unchanged.

The encoding is:

```
bytes 0-5    → unchanged
bytes 6-14   → +5
byte 15      → -3
bytes 16-25  → unchanged
```

So the suspicious ASCII data I found at the end of the PNG was indeed the encoded flag.

Reversing the Encoding



Since the binary adds 5 to bytes 6-14, we need to **subtract 5** to get the original characters back.

The binary subtracts 3 from byte 15, so we need to **add 3** to reverse it.

Everything else is unchanged.

Because the encoded flag is exactly 26 bytes, we can simply grab the last 26 bytes of the PNG:

```
with open("mystery.png", "rb") as f:
    flag = f.read()[-26:]

# Bytes 0-5: unchanged
for i in range(6):
    print(chr(flag[i]), end='')
```

```

# Bytes 6–14: originally +5, so subtract 5
for i in range(6, 15):
    print(chr(flag[i] - 5), end='')

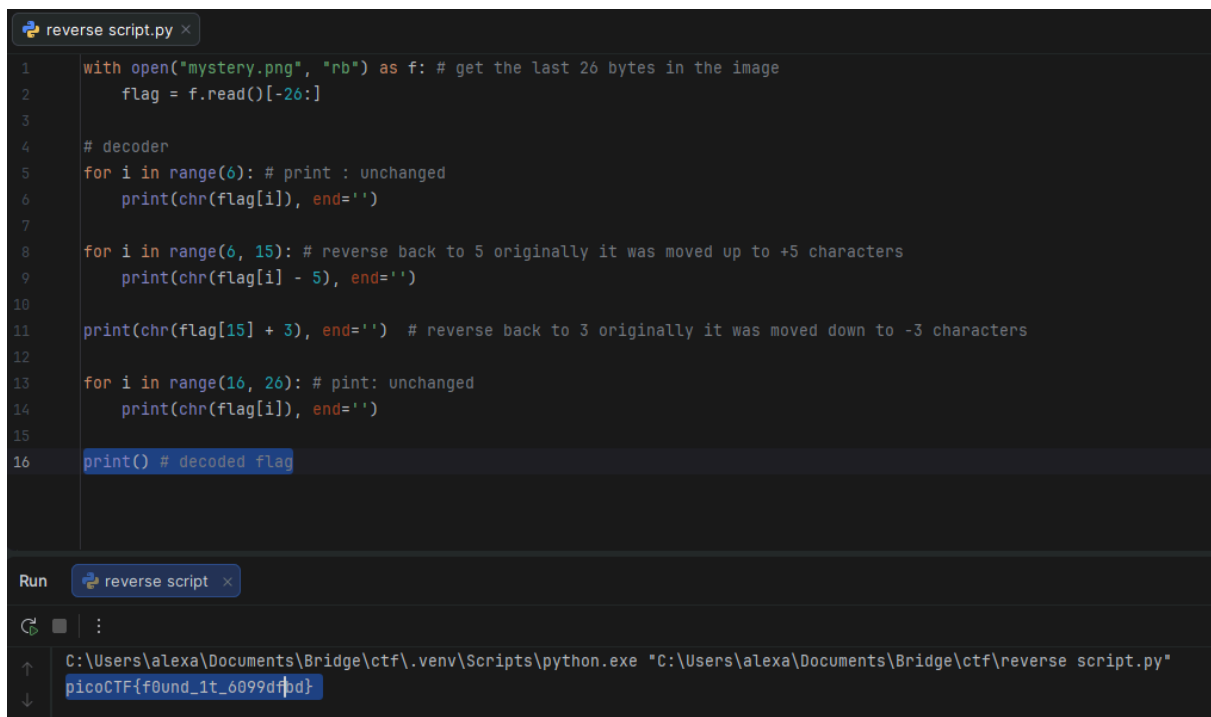
# Byte 15: originally -3, so add 3
print(chr(flag[15] + 3), end='')

# Bytes 16–25: unchanged
for i in range(16, 26):
    print(chr(flag[i]), end='')

print()

```

Running the script gives:



The screenshot shows a code editor with a file named 'reverse script.py'. The code reads the last 26 bytes of a file 'mystery.png' and decodes them. It applies a reverse shift of 5 to bytes 6-14, adds 3 to byte 15, and prints bytes 16-25 unchanged. The terminal output shows the command being run and the resulting flag: 'picoCTF{f0und_1t_6099dfbd}'.

```

reverse script.py x
1 with open("mystery.png", "rb") as f: # get the last 26 bytes in the image
2     flag = f.read()[-26:]
3
4     # decoder
5     for i in range(6): # print : unchanged
6         print(chr(flag[i]), end='')
7
8     for i in range(6, 15): # reverse back to 5 originally it was moved up to +5 characters
9         print(chr(flag[i] - 5), end='')
10
11     print(chr(flag[15] + 3), end='') # reverse back to 3 originally it was moved down to -3 characters
12
13     for i in range(16, 26): # print: unchanged
14         print(chr(flag[i]), end='')
15
16     print() # decoded flag

```

```

Run reverse script x
C:\Users\alexa\Documents\Bridge\ctf\.venv\Scripts\python.exe "C:\Users\alexa\Documents\Bridge\ctf\reverse script.py"
picoCTF{f0und_1t_6099dfbd}

```

picoCTF{f0und_1t_6099dfbd}

Flag

picoCTF{f0und_1t_6099dfbd}